RAPPORT PROJET TUTORÉ :

Temps d'attente en agences OPT-NC

Réalisé par :

Malcolm BERTAINA (L3T7 Informatique)

Morgan CARRE (L2T5 Informatique)

Sous la direction de :

Adriens SALES (OPT-NC)

Formation : Licence Informatique

Année : 2024

Table des matières

1. Remerciements **1**
2. Introduction **1**
3. Développement **1**
3.1. Rencontre & Pitch 1
3.2. Organisation et déploiement sur Streamlit Cloud 3
3.3. Tooling, stack technique : "Hello World" (v0.0) 4
3.4. MVP première version "fonctionnelle" (v0.1) 6
3.5. Historique et auto-refresh (v1.0) 8
4. Conclusion **12**
5. Références **13**

1. Remerciements

Nous tenons à remercier tout particulièrement Adrien Sales pour la proposition de ce projet captivant, ainsi que pour son suivi constant et ses précieux conseils, y compris en soirées et les week-ends, qui ont été déterminants pour le bon déroulement du projet.

Nous remercions également Michèle Barre et Vinh Faucher pour leur réactivité et leur soutien dans la résolution des bugs et des erreurs rencontrés avec l'API des temps d'attente.

2. Introduction

Dans le cadre de notre licence en informatique, nous avons eu l'opportunité de réaliser un projet tutoré. Nos professeurs nous ont proposé une liste variée de sujets, chacun présenté par des encadrants extérieurs à l'UNC, issus de différents secteurs et entreprises.

Intéressés tous les deux par le développement informatique, nous avons naturellement cherché un projet qui pourrait à la fois nous permettre de mettre en pratique nos connaissances et nous aider à développer de nouvelles compétences dans ce domaine.

Après avoir étudié les différents sujets, tous très intéressants, nous avons finalement décidé de nous concentrer sur le projet "interface web des temps d'attente en agence OPT", proposé par Adrien Sales de la DSI de l'OPT.

Ce sujet, qui aborde une vraie problématique du monde réel, nous semblait pertinent, d'autant plus qu'il impliquait une stack technique que nous n'avions encore jamais utilisée, offrant ainsi une excellente opportunité d'apprentissage.

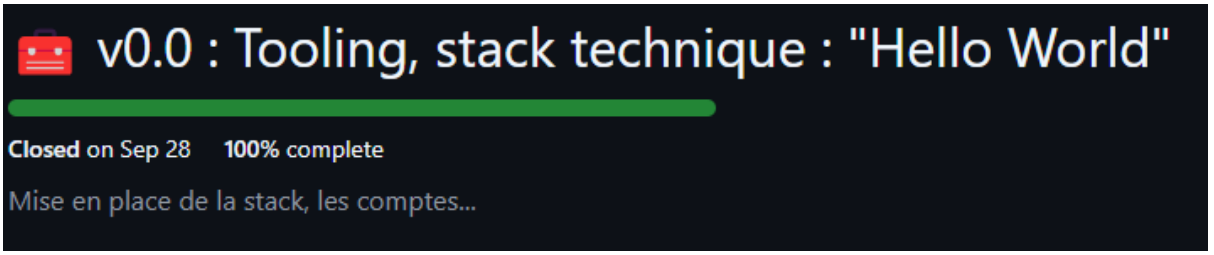
3. Développement

3.1. Rencontre & Pitch

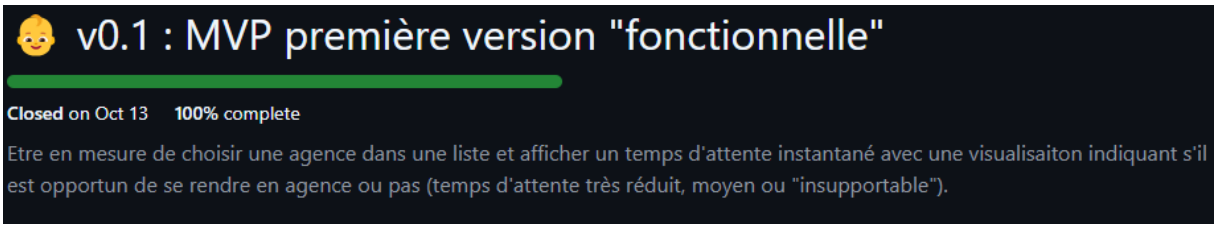
Le mercredi 18 septembre, nous avons rencontré Adrien Sales et Michèle Barre après un échange de mails. Lors de cette rencontre, Adrien a pitché le sujet du projet en détail et nous a présenté la méthodologie à adopter pour mener à bien le développement. Cette méthodologie reposait sur une organisation structurée en milestones pour jalonner les différentes phases du projet et en issues pour suivre les tâches à accomplir. Cette réunion a permis de clarifier le sujet du projet et d'en poser les bases.

Nous avons ainsi organisé une roadmap avec des objectifs bien définis et établi notre MVP (Minimum Viable Product). Le projet s'est structuré autour de trois grands objectifs :

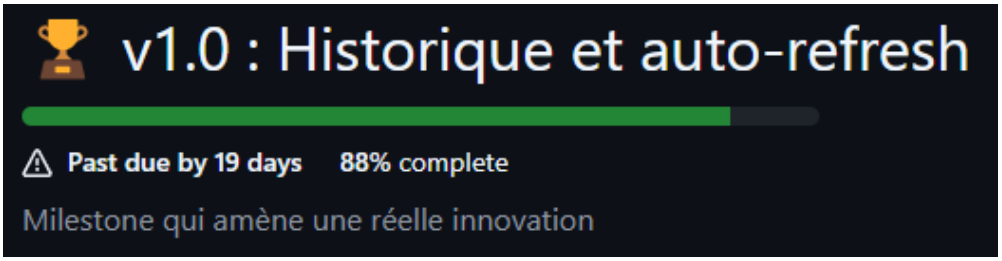
- Tooling, stack technique : "Hello World" (v0.0)

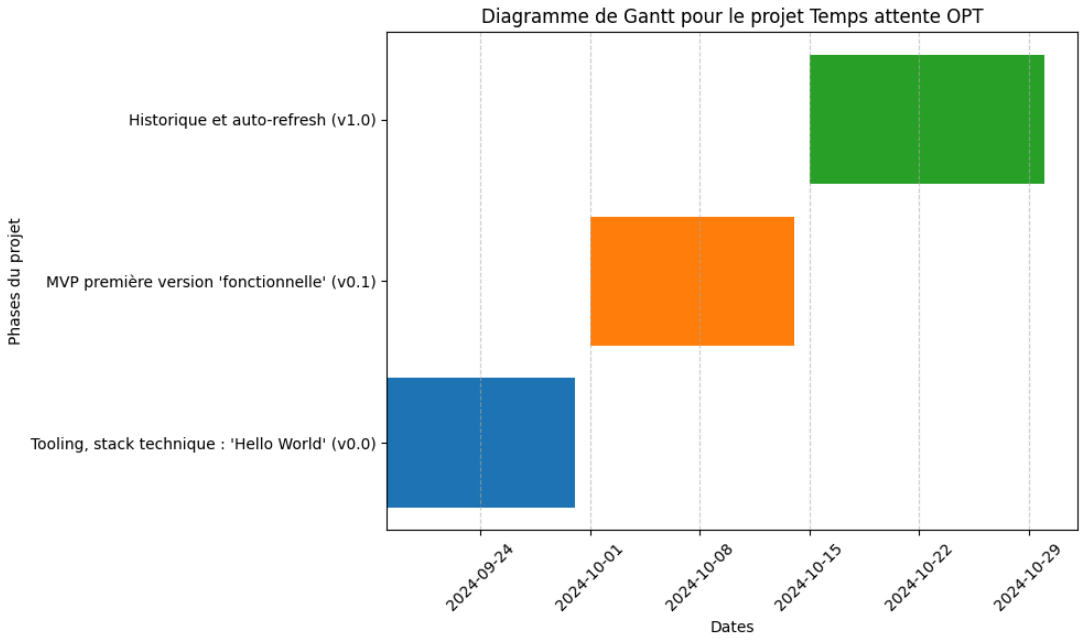


- MVP première version "fonctionnelle" (v0.1)



- Historique et auto-refresh (v1.0)





À la suite de cette rencontre, nous avons mis en place une collaboration sur GitHub (Milestones Ci-dessus), ce qui nous a permis de travailler efficacement et de rester en contact régulier avec nos encadrants. Il a également été décidé que Malcolm Bertaina assurerait le rôle de lead developer.

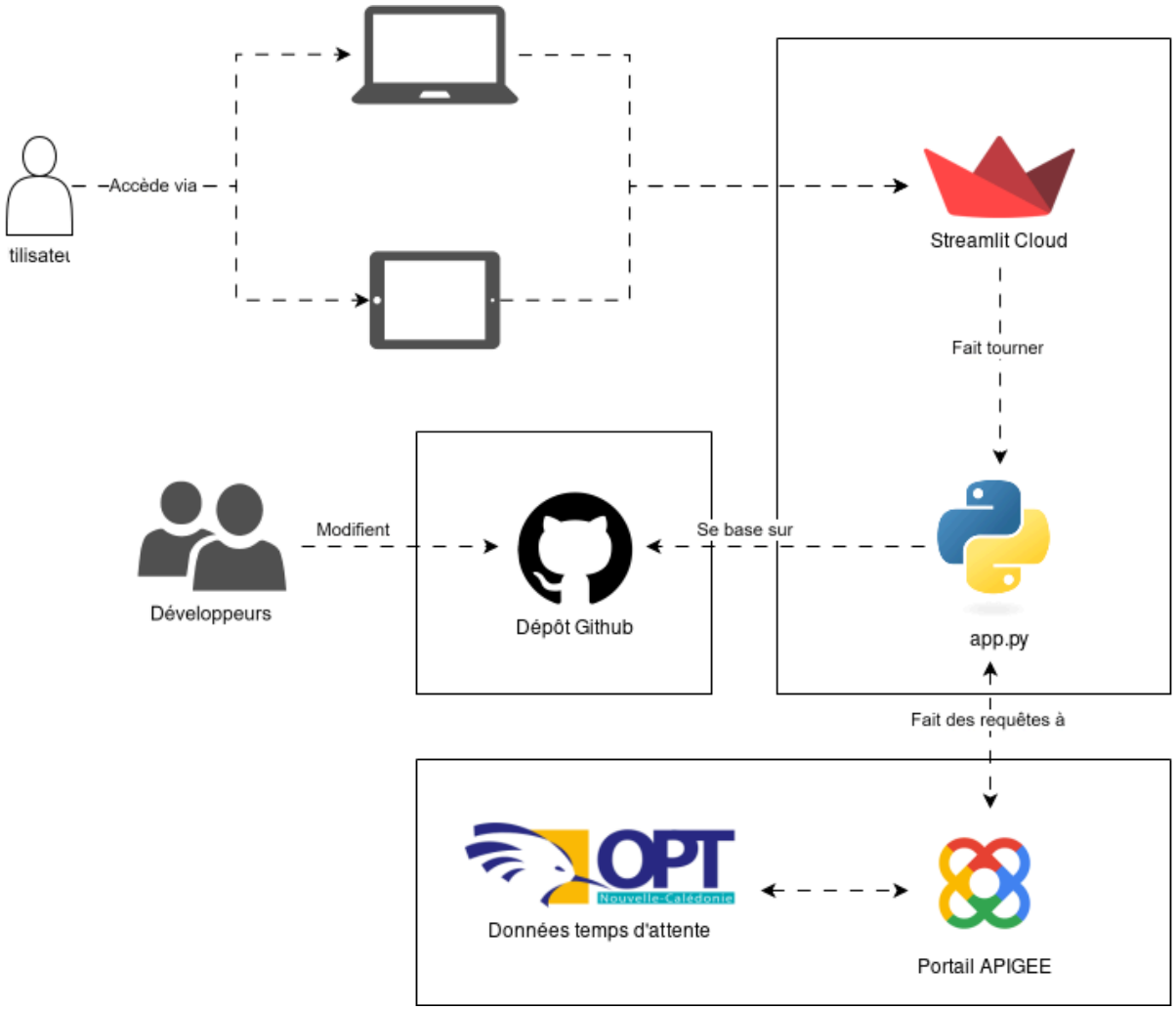
3.2. Organisation et déploiement sur Streamlit Cloud

La cinématique de gestion de notre projet sur GitHub et de déploiement sur Streamlit Cloud s'est déroulée en plusieurs étapes :

1. **Création d'issue** : Lorsqu'un problème ou une fonctionnalité à développer était identifié, nous devions créer une issue sur GitHub. Chaque issue était associée à un descriptif détaillant le problème à résoudre ou la tâche à accomplir. Cela nous a permis de se répartir plus facilement les différentes tâches.
2. **Création d'une branche** : Pour chaque issue, la personne qui y était assignée devait créer une nouvelle branche pour ne pas directement travailler sur main, la branche principale.
3. **Création de la pull request (PR)** : Une fois le développement effectué sur une branche dédiée, nous créions une pull request (PR) pour demander de merge notre travail vers la branche principale. La PR incluait une description de la fonctionnalité ou de la correction, ainsi qu'une référence à l'issue que fermait cette PR.
4. **Revue de code** : Après la création de la PR, notre tuteur effectuait une revue de code. Cette étape était cruciale pour vérifier qu'il n'introduisait pas de bugs. Lors de cette revue, des suggestions d'améliorations pouvaient être faites, et le code pouvait être modifié avant de merge.
5. **Merge de la PR** : Une fois la revue terminée et validée, la PR était fusionnée dans la branche principale, ce qui permettait d'intégrer la fonctionnalité ou la correction au projet principal.
6. **Déploiement sur Streamlit Cloud** : Après chaque merge dans la branche principale, un déploiement automatique sur Streamlit Cloud était effectué. Ce service hébergeait notre application et la mettait à jour à chaque nouvelle version.

Cette dernière étape, réalisée par Adrien, a permis de déployer notre application à l'adresse opt-nc-temps-attente.streamlit.app. Le service offre l'avantage de rendre le projet accessible en ligne de manière rapide et gratuite, avec une mise à jour automatique après chaque modification. Ainsi, tous les membres de l'équipe et les utilisateurs externes pouvaient tester et utiliser l'application en temps réel, sans nécessiter de déploiements manuels à chaque changement. Ce déploiement facilite également une démonstration en direct lors de la soutenance.

Pour mieux comprendre l'organisation technique de notre solution, ci-dessous un schéma représentant l'architecture globale de l'application.



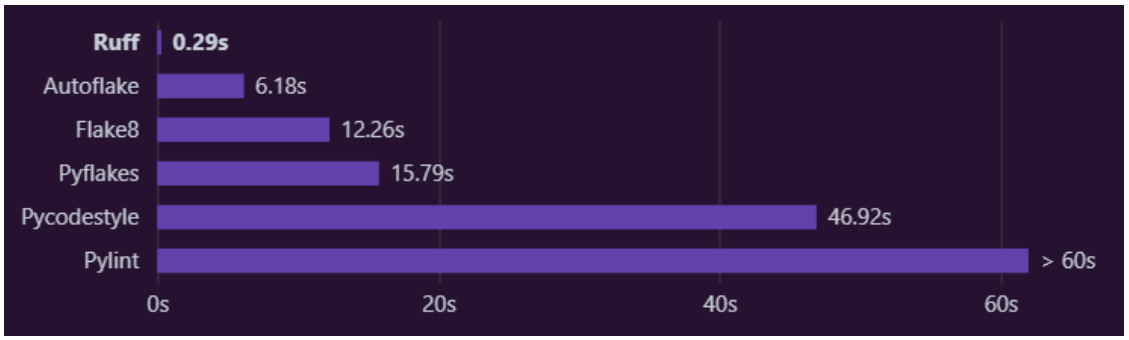
3.3. Tooling, stack technique : "Hello World" (v0.0)

La version v0.0 a constitué la première étape de notre projet. Elle s'est concentrée sur la mise en place de l'infrastructure de développement, des outils de collaboration, et sur la sélection de la stack technique. Elle a également inclus le déploiement d'une première version minimaliste "Hello World" afin de valider l'ensemble des choix techniques et de tester le bon fonctionnement de notre environnement.

La stack que nous avons choisie pour ce projet est Streamlit en Python, sélectionnée pour sa flexibilité dans le développement rapide d'applications web interactives et sa compatibilité avec de nombreuses bibliothèques de traitement de données. Le développement a été réalisé principalement sous Ubuntu, avec des tests sur différents environnements, notamment Windows et Ubuntu. Le déploiement a été géré via Docker, ce qui nous a permis de garantir une portabilité optimale.

Pour garantir une qualité de code optimale et faciliter la collaboration, nous avons commencé par mettre en place un linter. Nous avons opté pour Ruff, un linter

performant et rapide, connu pour sa faible consommation de ressources et sa capacité à détecter un large éventail d'erreurs courantes dans le code Python. Ruff a ainsi permis d'assurer une cohérence dans notre style de code, de détecter rapidement les erreurs potentielles, et de maintenir une base de code lisible et de qualité.

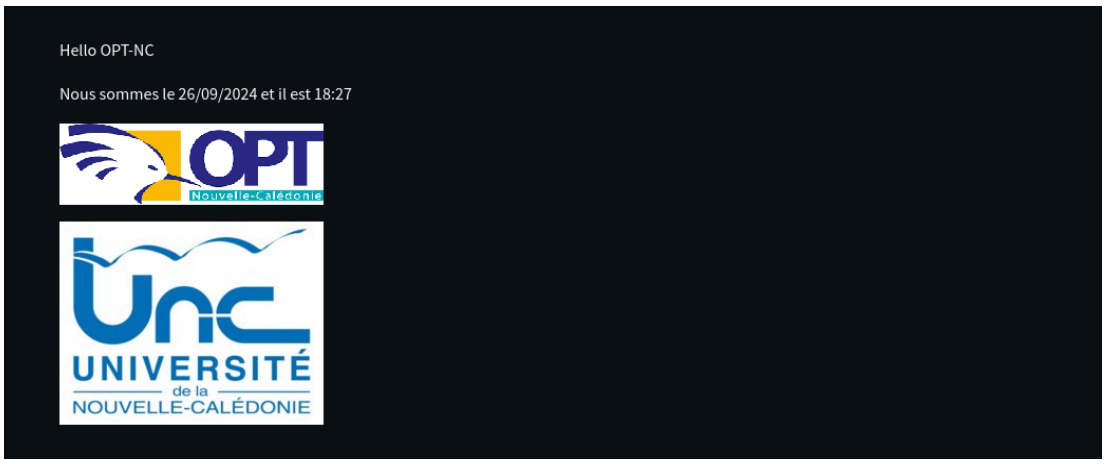


Nous avons également mis en place un social preview afin d'améliorer l'affichage lors du partage sur les réseaux sociaux ou autres plateformes, contribuant à donner une image plus professionnelle et engageante de l'application.



Ensuite, nous avons créé nos comptes sur le portail d'API de l'OPT (APIGEE), afin d'obtenir les informations sur les agences et les temps d'attente en temps réel, des données essentielles au bon fonctionnement de notre application.

Enfin, nous avons lancé la première version de notre projet, qui se composait d'une page affichant "Hello World OPT-NC" accompagnée des logos des établissements UNC et OPT. Cela nous a permis de vérifier que l'ensemble des éléments mis en place fonctionnait correctement. Nous avons créé une image Docker pour garantir la portabilité, la reproductibilité et la facilité de déploiement de notre projet. La documentation a ensuite été mise à jour, détaillant la procédure de démarrage de l'application afin de faciliter son utilisation pour les futurs contributeurs et utilisateurs.



3.4. MVP première version "fonctionnelle" (v0.1)

Dans ce milestone, l'objectif était de créer une première version viable de l'application capable d'interagir avec l'API. Dans cette version initiale, l'utilisateur devait pouvoir sélectionner une agence dans une liste déroulante et voir le temps d'attente instantané sous forme de jauge, semblable à un compteur de voiture.

Dans un premier temps, nous avons simplement affiché le temps d'attente sous forme de texte, afin de nous concentrer sur l'interaction avec l'API. Pour ce faire, nous avons utilisé différents endpoints :

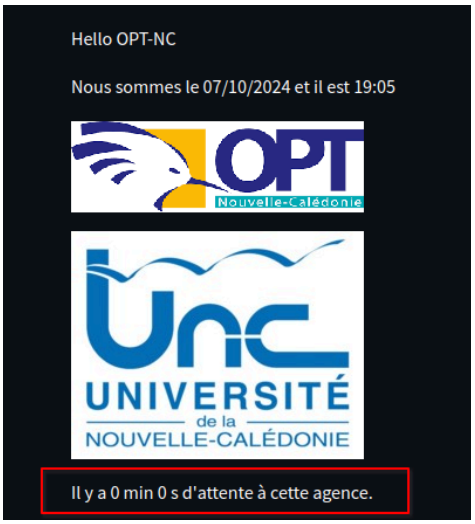
- **/communes** : pour récupérer une liste des communes et en faire une liste déroulante.
- **/agences?commune=[nom_commune]** : pour obtenir la liste des agences en fonction de la commune sélectionnée, afin de les afficher dans une deuxième liste déroulante.

L'endpoint **/agences?commune** nous a permis de récupérer de nombreuses informations sur chaque agence d'une commune, dont leur temps d'attente en temps réel.

Comme nous pouvons le voir ci-dessous avec l'usage de l'outil HTTPie :

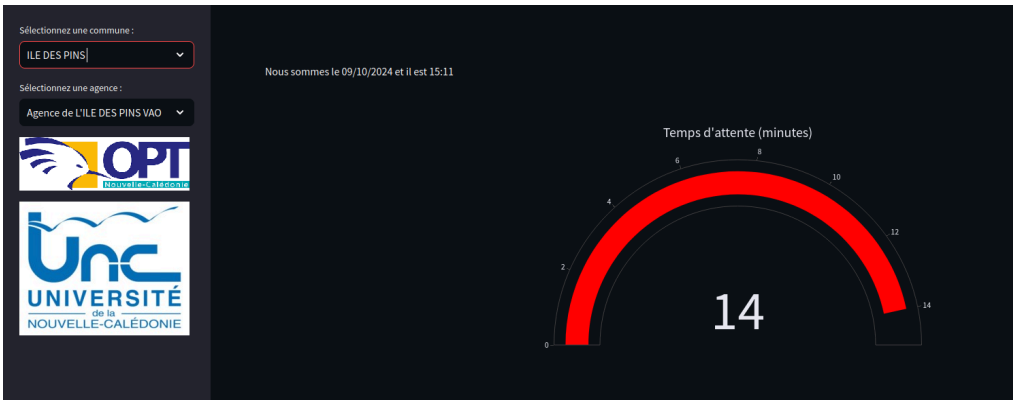


Nous avons donc commencé par afficher cette donnée sous forme de texte pour valider le bon fonctionnement de l'API.



Ensuite, nous avons ajouté la jauge comme prévu dans le milestone. Pour cela, nous avons utilisé la bibliothèque Plotly, car elle est simple à intégrer dans Streamlit via la méthode plotly_chart(). Elle offre également des visualisations interactives et adaptables, ce qui la rend pratique pour des graphiques dynamiques et clairs. La jauge permet ainsi aux usagers de voir instantanément l'état d'affluence des agences. En fonction du temps d'attente t, la couleur de la jauge change :

- Si t est inférieur ou égal à 5 minutes, la jauge est verte.
- Si t est compris entre 5 et 10 minutes, la jauge passe au orange.
- Si t est supérieur à 10 minutes, la jauge devient rouge.



Plusieurs corrections de bugs et optimisations ont également été réalisées dans cette version :

- La clé API a été placée en variable d'environnement pour renforcer la sécurité et éviter son exposition dans le code.
- Les logos de l'OPT et de l'UNC ont été déplacés dans la sidebar, afin de désencombrer la page principale et améliorer l'interface.
- La timezone a été ajustée pour correspondre à celle de la Nouvelle-Calédonie, car lors du déploiement sur les serveurs de Streamlit, la configuration par défaut n'était pas correcte.

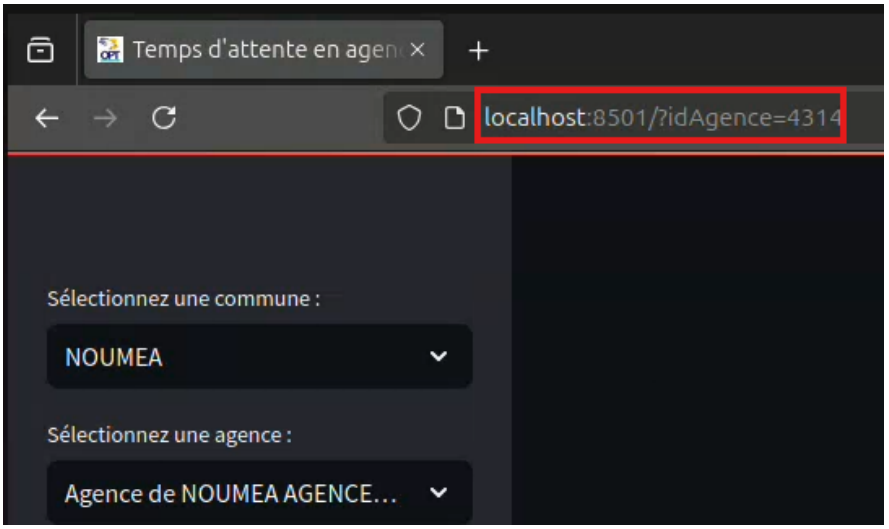
3.5. Historique et auto-refresh (v1.0)

Cette milestone V1 apporte une grosse innovation sur le projet. Elle a pour but d'améliorer l'expérience utilisateur en ajoutant un historique des temps d'attente sur la journée pour que celui-ci puisse anticiper la fréquentation des agences en se basant sur les tendances passées.

Dans un premier temps, nous avons travaillé sur l'optimisation et la refactorisation du code existant. Désormais, les appels à l'API sont gérés via des fonctions définies dans le module utils.py, telles que **fetch_communes()** et **fetch_agences(commune)**. Les données récupérées sont mises en cache avec un Time-To-Live (TTL) grâce au décorateur **@st.cache_data()**, afin de minimiser les appels redondants à l'API lorsque les informations restent inchangées. Une constante **INTERVALLE_AUTOREFRESH** a également été ajoutée pour rendre explicite le délai d'auto-rafraîchissement soit 3 minutes.

Par ailleurs, plusieurs corrections mineures ont été apportées, qu'il s'agisse d'améliorations visuelles ou de fonctionnalités :

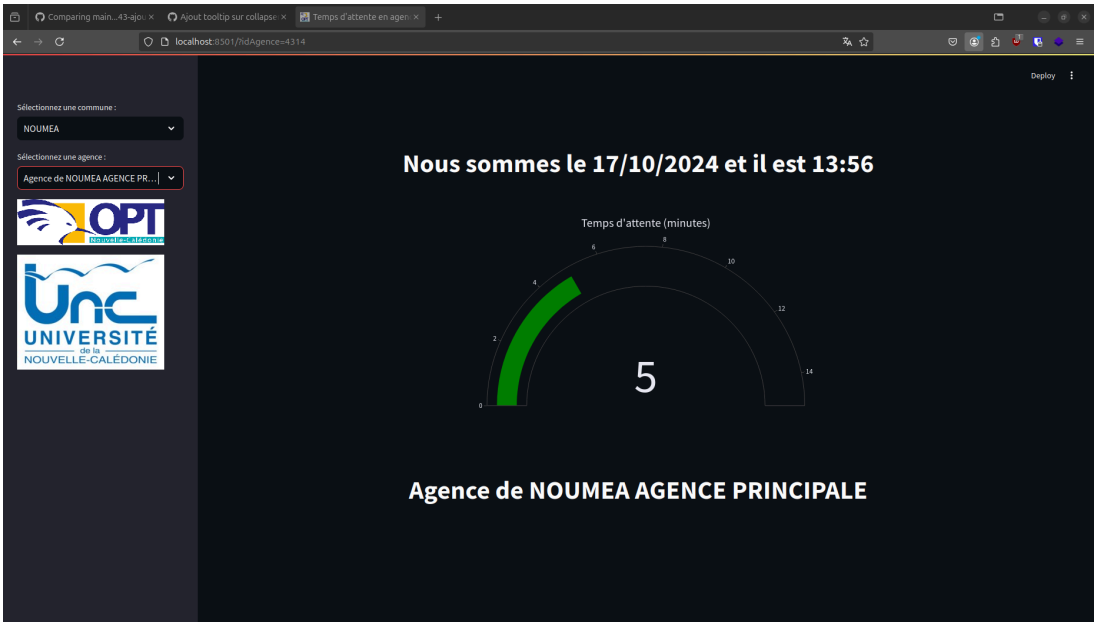
- Ajout de l'id de l'agence dans l'URL pour permettre d'avoir accès directement au temps d'attente d'une agence grâce à l'URL et adapter le menu déroulant en conséquence. L'agence correspondante est donc pré-sélectionnée lors de l'accès via l'URL. Pour ce faire, une nouvelle fonction **fetch_agence_by_id(id_agence)** récupère les informations spécifiques à l'agence passée en paramètre.



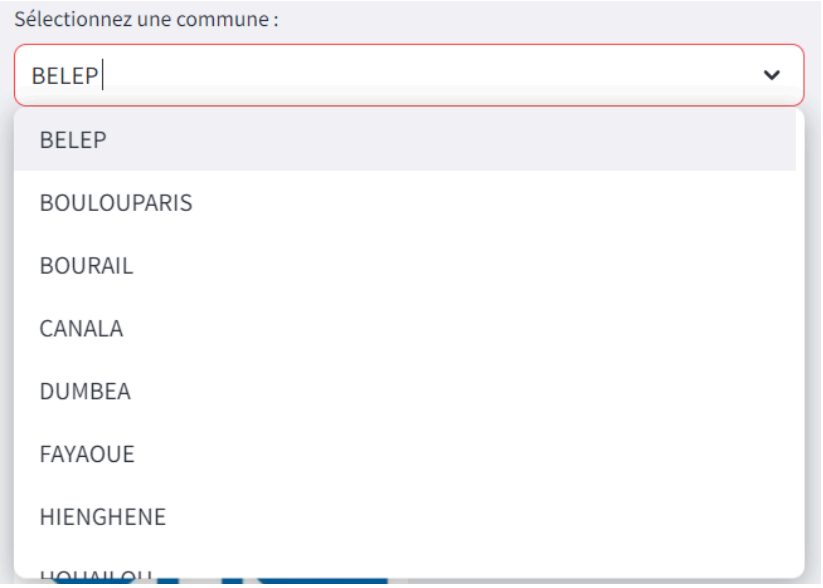
- Ajout d'un élément visuel pour indiquer que l'on peut fermer ou ouvrir la barre latéral



- Affichage du nom de l'agence : Le nom de l'agence s'affiche désormais sous la jauge de temps d'attente grâce à un style personnalisé.



- Tri alphabétique des communes : La liste des communes est triée par ordre alphabétique à l'aide de **sorted()** pour une meilleure lisibilité.

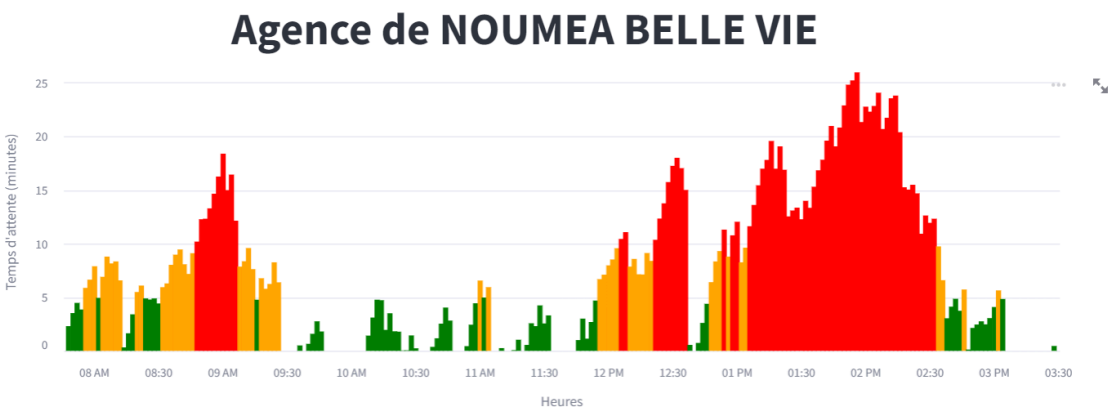


Ensuite, nous avons abordé la fonctionnalité principale de cette version : l'historique des temps d'attente d'une agence sélectionnée.

Pour cela, nous avons utilisé l'endpoint **temps-attente-agences/agences/{id}/historique**, qui renvoie les temps d'attente espacés de 2 minutes pour une agence donnée sur une période spécifiée. Cependant, nous avons rencontré un problème lors de son utilisation : l'API renvoyait une erreur. Après avoir signalé ce bug à Vinh, membre de l'équipe d'Adrien, il a confirmé qu'il s'agissait d'un dysfonctionnement de leur côté. En attendant la correction, il nous a orientés vers deux portails alternatifs où l'API fonctionnait correctement : **RapidAPI** et **Killercoda**. Nous avons opté pour RapidAPI.

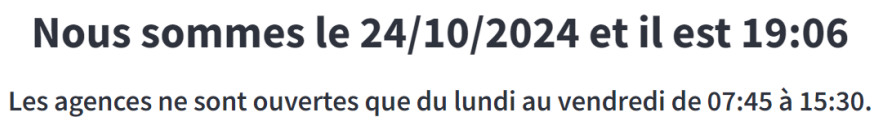
Comme nous utilisions désormais deux portails différents (APIGEE et RapidAPI), il a été nécessaire d'ajouter l'URL **https://temps-attente-en-agence.p.rapidapi.com/** ainsi que la clé associée dans le fichier **.env**.

Une fois l'environnement mis à jour, nous avons pu récupérer l'historique des temps d'attente via une nouvelle fonction, **fetch_agence_historique(id_agence, debut, fin)**, qui renvoie les données sous forme de dataframe (librairie **Pandas**). Ce dataframe est ensuite utilisé pour l'affichage grâce à **st.altair_chart()**, en optant pour un histogramme avec des couleurs spécifiques selon le temps d'attente (vert : $t < 5$ min, orange : 5 min $\leq t < 10$ min, rouge : $t > 10$ min) pour une lisibilité optimale.



Une difficulté supplémentaire a été rencontrée au niveau des fuseaux horaires : l'API fonctionnant en UTC, alors que nous devions utiliser le fuseau UTC+11 (Nouvelle-Calédonie). Nous avons corrigé ce problème en appliquant un décalage de -11 heures aux paramètres debut et fin de la fonction fetch_agence_historique() grâce à **timedelta**.

Pour finaliser cette milestone, nous avons ajouté un message indiquant les horaires d'ouverture des agences OPT. Si un utilisateur accède à notre application en dehors de ces horaires (par exemple, un dimanche), il est informé que les agences sont fermées et se voit fournir les horaires d'ouverture. Cette fonctionnalité repose sur la fonction **check_valid_hours(start, end)** qui vérifie si le jour actuel n'est pas un samedi ou un dimanche et si l'heure est bien comprise entre 7h45 et 15h30, en se basant sur **get_current_time()**.



4. Conclusion

Ce projet nous a permis de consolider nos compétences techniques tout en découvrant de nouvelles technologies. La collaboration sur **GitHub** a été un excellent moyen de renforcer notre maîtrise des pratiques de travail en équipe, notamment avec les branches, les pull requests et les revues de code.

Nous avons également approfondi notre utilisation de **HTTPie** pour tester et valider les endpoints de l'API. Cela a été essentiel non seulement pour assurer le bon fonctionnement de notre application, mais aussi pour identifier rapidement un bug lié à l'API de l'OPT, ce qui a facilité le **bug reporting** auprès de notre tuteur.

Un autre point marquant a été la découverte du framework **Streamlit**, que nous avons appris à utiliser pour créer rapidement une interface web interactive avec des visualisations dynamiques.

En résumé, ce projet nous a permis de combiner apprentissage technique et travail collaboratif tout en relevant un défi concret, nous préparant ainsi à aborder des projets professionnels avec plus de confiance et d'efficacité.

5. Références

"Streamlit Documentation." Streamlit, https://docs.streamlit.io/. Accédé le 13 novembre 2024.

"Ruff - Fast Python Linter." Astral, https://docs.astral.sh/ruff/. Accédé le 13 novembre 2024.

"Portail APIGEE de l'OPT NC." OPT-NC, https://apigee-optnc-prd-api.apigee.io/. Accédé le 13 novembre 2024.

"Plotly for Python - Interactive Graphing." Plotly, https://plotly.com/python/. Accédé le 13 novembre 2024.

"Streamlit Caching." Streamlit Documentation, https://docs.streamlit.io/develop/concepts/architecture/caching/. Accédé le 13 novembre 2024.

"Portail RapidAPI de l'OPT NC." OPT-NC, https://rapidapi.com/opt-nc-opt-nc-default/api/temps-attente-en-agence/. Accédé le 13 novembre 2024.

"Portail Killercoda de l'OPT NC." OPT-NC, https://killercoda.com/opt-labs/course/opt-temps-attente-agences/. Accédé le 13 novembre 2024.

"HTTPie" HTTPie, https://httpie.io/docs/. Accédé le 13 novembre 2024.